

ACTIVE-IA

Devolución de Trabajo Práctico

Materia:	PROG3-M2026 - Programación 3
Comisión:	COMI-15 (2026)
Trabajo:	Trabajo Práctico - ToString-Colecciones
Alumno:	Macarena Marinoni

Calificación Final: 98/100

EVALUACIÓN POR CRITERIOS

C1: Desarrollo de Clases y Relaciones (UML)

30/30



Todas las clases y relaciones están correctamente implementadas según el diagrama UML. Se respetan atributos, métodos, enumeraciones, herencia y el uso estricto de colecciones tipo Set en la clase Pedido, incluyendo la lógica para manejar la unicidad de los detalles.

C2: Contrato de Objetos: toString, equals y hashCode

30/30



Todos los métodos toString(), equals() y hashCode() han sido sobrescritos en cada clase. La coherencia de identidad se mantiene correctamente (ej. Producto por nombre, Usuario por mail, Pedido por ID), y los métodos toString() evitan la recursión cíclica, mostrando información relevante.

C3: Instanciación y Escenario de Datos (Main)

20/20



El escenario de objetos solicitado en la clase Main está creado de forma precisa y completa. Se instancian correctamente los usuarios, categorías, productos y pedidos, con la cantidad de detalles y asignaciones de usuario especificadas.

C4: Visualización y Procesamiento por Consola

10/10



La visualización por consola es clara y cumple con todos los requisitos. Se utiliza el método `toString()` de manera efectiva para mostrar productos individuales, listados de productos y los pedidos del usuario con más órdenes, incluyendo sus detalles.

C5: Validación Práctica de Colecciones y Equivalencia

8/10



La lógica de unicidad del Set está correctamente implementada en la clase Pedido (método `addDetallePedido`), demostrando una excelente comprensión. Sin embargo, la prueba explícita en el Main para el punto 5 utiliza un `ArrayList`. Aunque `ArrayList.contains()` usa `equals()`, la rúbrica pedía una prueba de la unicidad del Set, lo que podría haberse demostrado más directamente con una colección Set en el Main.

FORTALEZAS

- Implementación robusta de los contratos `toString`, `equals` y `hashCode` en todas las entidades, manejando correctamente la identidad lógica y la prevención de recursión cíclica.
- Uso correcto y efectivo de la colección Set en la clase Pedido para gestionar los `DetallePedido`, demostrando una comprensión de la unicidad y la actualización de elementos.
- El escenario de datos en Main está completo y bien estructurado, facilitando la prueba y demostración de las funcionalidades.
- La visualización por consola es clara y utiliza adecuadamente los métodos `toString` para presentar la información de los objetos.

RECOMENDACIONES

1. Para el punto 5, se recomienda realizar la prueba de unicidad directamente con una colección de tipo Set en el Main para una demostración más explícita de su comportamiento.
2. Considerar el uso de `Optional` para el método `findDetallePedidoByProducto` en la clase Pedido para manejar de forma más idiomática la ausencia de un detalle.
3. Asegurar que los IDs de los `DetallePedido` se gestionen de alguna manera (aunque sea con un contador simple en `addDetallePedido`) en lugar de pasar null en el constructor.

COMENTARIOS DEL EVALUADOR

El trabajo de Macarena Marinoni es excelente, demostrando una sólida comprensión de los contratos de objetos y el manejo de colecciones. La implementación de `toString`, `equals` y `hashCode` es ejemplar, y la lógica de negocio en la clase `Pedido` para gestionar los detalles con un `Set` es muy acertada. Con algunas mejoras menores en la demostración del `Set` en `Main` y el uso de `Optional`, el trabajo sería impecable.